

Document 14 — Debugging Lab (covers everything)

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

Docs 01-13 (this practices everything you've built, together).

How to Read & Practice This Document

- **What:** a repeatable way to figure out failures, instead of guessing.
- **Why:** real AI systems fail in layered, unclear ways — changing things randomly until it works doesn't scale past small projects.
- **When:** every single time something breaks, from here through your real job.
- **How to practice:** this document is live and interactive on purpose — there's no solo build task. Read the method below, have Projects 1-4 working, then say **START DOCUMENT 14** and work through each planted bug using the steps below, out loud, before I confirm or correct your answer.

Jump to: [Core Concepts](#) · [Practice Exercises](#)

The Story — what this document is actually building

Every document before this one added a new skill: functions and config (Doc01), API calls, tools, RAG, LangGraph, multi-agent orchestration, production hardening, and a real test suite (Doc13). Each one, on its own, worked by the time you moved on. This document is where all of that gets stress-tested at once — because a real system doesn't fail one clean layer at a time, it fails in ways that *look* like they could be any of five different things at once.

First, you need a fixed way to tell a symptom fix from a real fix. Silencing an error with a bare `try/except`, or hardcoding a value that happened to fail once, makes the failure in front of you disappear — without you ever learning why it happened, which means it comes back later, usually more confusing the second time.

Second, you need to be able to read an error trace without panicking — it's not noise, it's an ordered map of exactly which calls were running when things broke, and reading it from the bottom up gets you to the real spot faster than scanning it for anything that looks familiar.

Third, you need a *reliable* way to reproduce a failure before you can test any guess about it — a fix you can't verify isn't really a fix, it's a hope.

Fourth, you need to check failures layer by layer, starting low (was the right tool even called? was its input correct?) instead of starting at the final answer — because in a multi-agent system, the final answer is the layer furthest from the actual cause, and the least useful place to start looking.

That's the whole story: this document doesn't teach new libraries or new code — it teaches **the Method**, a fixed sequence (Observe → Reproduce → Guess → Test → Real Cause → Fix → Test Again → Prevent) applied to real bugs planted into the real projects you already built. There's no Build Task here on purpose — this document *is* the practice, run live, against Projects 1-4, until debugging this way is a habit instead of a checklist.

The Method (learned here, used everywhere after)

```
Observe → Reproduce → Form a Guess → Test the Guess
→ Find the Real Cause → Fix → Test Again → Prevent It Happening Again
```

No "just Google the error." Every planted bug below gets worked through this way, out loud, before I confirm or correct your answer.

Core Concepts (read this first — everything you need is here)

Why "real cause" and "just the symptom" are different things

A symptom fix makes the failure you're seeing stop happening right now (adding a `try/except` that just swallows the error, hardcoding a value that happened to fail once) without dealing with *why* it happened — so the same underlying problem will show up again, often in a more confusing form. A real-cause fix deals with the actual reason the failure happened, checked by actually understanding it — not just by the symptom going away. **Why this difference is the whole point of the method:** it's very easy to change code randomly until one specific test passes, call it fixed, and leave a real bug in place, just hidden better. The method exists specifically to stop that.

Reading an error trace as a map, not noise

A Python error trace lists the exact chain of function calls that were running when the error happened, in order — the *last* part is where the error actually occurred; the parts above it are the calls that led there. **Why this matters:** panic-scrolling a long error trace looking for anything familiar

is slower than reading it in order from the bottom up — find the exact line and error type, then work upward only as far as you need, to understand *why* that line got bad input in the first place.

Reproduce before you guess

Before forming any guess about what's wrong, you need a *reliable* way to make the failure happen again — a guess tested against a bug you can't reliably reproduce can't really be tested, since you can't tell whether a "fix" actually worked, or the bug just didn't happen to trigger that time. **Why bugs that only sometimes happen** (a test that fails randomly, an agent that sometimes loops) **are harder specifically because of this**: the first real work isn't fixing anything, it's narrowing down *exactly what condition* makes the failure reliable enough to study — timing, one specific input, one specific sequence of tool calls.

Debugging systems with many layers

An AI agent system can fail across several layers that look identical from the outside — a wrong final answer could be a plain Python bug, an API error being quietly swallowed, a bad tool description causing the wrong choice, a chunking problem in search, or a state-handling bug in the graph. **Why checking layer by layer matters more here than in normal software**: the highest layer (the final answer) is the *least* useful place to start — it's better to check, in order, whether the right tools were called, whether their inputs/outputs were correct, whether the graph routed the way you expected, and only then ask whether the model's own reasoning was actually wrong, given genuinely correct inputs.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail. - [Python docs — pdb, the debugger](#) — stepping through code beats guessing from `print()` statements. - Read [Anthropic — Building Effective Agents](#) again, especially any part on failure — you now have enough built to recognize every example.

What You'll Practice

This document has no new library or tool content — it's a practiced, well-known method, applied to every kind of failure from Docs 01-13: - **Python**: import errors, type errors, exceptions, async mistakes, package version conflicts. - **API**: bad key, timeout, rate limit, bad request, bad JSON, login errors. - **OpenAI**: too-long conversation, bad structured output, tool-call failure, using too many tokens, model errors. - **LangChain**: a badly built chain, tool failure, parser failure, context problems. - **RAG**: bad chunking, bad search results, wrong documents shown, missing context, made-up answers. - **LangGraph**: wrong state, wrong edge, endless loop, bad routing, node failure, checkpoint problems. - **Multi-agent**: wrong agent picked, agents disagreeing, an endless loop

between agents, repeated tool calls, wrong state sharing, one agent's notes leaking into shared state, high cost, high delay.

Practice Exercises — how each round works

Where the code lives: no separate practice files for this document — bugs get planted directly into Projects 1-4's real code, inside their own project folders (`project_1_supportdesk_chat_and_triage/` , etc.). Nothing to set up ahead of time beyond having those projects working.

For each planted bug, you get: broken requirements, symptoms, error/log output, what should happen vs. what's actually happening. Then: **"What do you think is wrong?"** — you look into it and answer, before I tell you if you're right.

Round structure (repeated across the categories above): 1. **Basic** — a single-layer bug (like a plain Python `TypeError`), quick to find. 2. **Intermediate** — a bug that crosses two layers (like an API error looking like a parsing bug). 3. **Real-world** — a bug pulled from a realistic, production-style situation (like something that only sometimes happens). 4. **Multi-agent** — a bug that's only visible in the full Project 4 pipeline, not when testing any one agent alone.

Try one yourself first — worked examples

This document has no fixed exercise list — bugs get planted live. Before your first live session, each category below has one fully worked example, at all 4 round levels, so you can practice the Method solo first.

Jump to a category: [Python](#) · [API](#) · [OpenAI](#) · [LangChain](#) · [RAG](#) · [LangGraph](#) · [Multi-agent](#)

Python

- **Python:** [See a worked example](#) · [Show me the solution](#)

API

- **API:** [See a worked example](#) · [Show me the solution](#)

OpenAI

- **OpenAI:** [See a worked example](#) · [Show me the solution](#)

LangChain

- **LangChain:** [See a worked example](#) · [Show me the solution](#)

RAG

- **RAG:** [See a worked example](#) · [Show me the solution](#)

LangGraph

- **LangGraph:** [See a worked example](#) · [Show me the solution](#)

Multi-agent

- **Multi-agent:** [See a worked example](#) · [Show me the solution](#)

Build Task — none

This document doesn't add new code to your projects. It stress-tests everything you've already built. Bring Projects 1-4 (and the Doc12/13 API + test layer) working — bugs get planted *into* them, live, during our sessions.

Expected Behavior

- You can state a testable guess before touching any code, not just start changing things at random.
- You can tell the difference between "I fixed the symptom" and "I found the real cause" — and explain which one you did.
- After a fix, you write (or update) a test that would have caught the bug — this step isn't optional.

Test Cases

This document's "test cases" are the planted-bug rounds themselves, done live during our sessions — not something to prepare ahead of time. When you say you're ready for this document, I'll plant bugs across the categories above, one round at a time, in your actual code.

Interview Topics Preview

- Walk through a real debugging session, start to finish, out loud, as if I'm interviewing you · real cause vs. symptom fix · how you'd debug something you can't reproduce locally.

Move On When

You can work through at least 5 planted bugs across different layers (Python/API/LangGraph/multi-agent) using the method above, on your own, without me nudging you toward the answer. Full

details: [CURRICULUM.md §4](#).

This document runs live. Say **START DOCUMENT 14** when Projects 1-4 are working and you want bugs planted into them.